

Simplex Reflex Agent

Xinyang Zhang

Understanding

First, I reviewed the PPT and the notes what I took in the class, I saw there are some important parts I need to remind.

Environment Framework

The PPT give the definition of the PEAS (Performance, Environment, Actuators, Sensors) :

1. **Performance:** I consider to use two kinds of scores, Using performance_score evaluates the cleanliness score and the consumption through energy_usage. I think it's closer to actual design considerations.
2. **Environment:** Including 2 rooms (A and B), and their cleaning status ('Dirty' or 'Clean').
3. **Actuators:** Three actions are implemented - Suck (clean), Left (move left), Right (move right).
4. **Sensors:** the agent can detect whether the current room is dirty. If I consider to use a Object-Oriented Principles considerations, I need to consider a method to do this step.

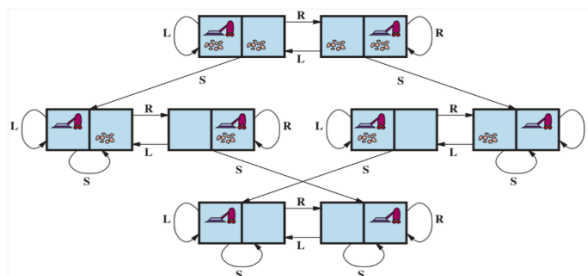
Rational agent

The agent should act based on actions chosen by percept:

1. If the current room is dirty, prioritize cleaning
2. When the front room is clean, choose the correct direction of movement

Initial configuration

Totally, there are 8 kinds of states. I use initial_states and initial_locations to iterate through all possible configurations, ensuring code coverage and generality.



Performance Analysis

Performance Score

1. **Clean score:** 1 point for each dirty square cleaned (by performing a Suck action).
2. **Extra points** for clean squares: 1 point for each clean square per time step.
3. Ensure that cleaning actions are prioritized and scores are recorded correctly.

Energy Usage

1. Consumption should be recorded for each action (including cleaning and moving).
2. Analyze the agent's energy performance under different initial conditions to avoid unnecessary movements.

Code

In Simple Reflection Proxy Implementation of Vacuum World, I modeled 4-time steps.

```
# Simple Reflection Proxy Implementation of Vacuum World
# Author: Xinyang Zhang
# Date: January,22 2025

import matplotlib.pyplot as plt

# setting environment
# To make the code cleaner and neater, I use Objective in the Python programming language.
# Totally there are three objects in this condition
# 1. environment
# 2. actuator
# 3. SimpleReflexAgent
class VacuumWorld:
    def __init__(self, initial_location="A"):
        # Initialize the state of the environment, the dirty or clean state of the A and B
        locations.
        # It is just a default setting, I will set complete configuration states in the main
        function

        self.states = {
            "A": "Dirty",
            "B": "Dirty"
        }

        # Initial position of the vacuum cleaner, can be set to A or B
        self.agent_location = initial_location
        self.performance_score = 0 # Performance scores
        self.energy_usage = 0 # Energy Consumption Score

        # Sensor,this part I put into environment, because
        def is_dirty(self):
            # Determine if the current vacuum cleaner location is dirty
            return self.states[self.agent_location] == "Dirty"

# Actuator action
# In order to make the code neater, I implemented the executor as a separate object
# I set 2 kinds of performance scores:
# 1.Add 1 performance score for each cleanup
# 2.Clearing actions and moving actions consume energy, adding energy usage scores.
class Actuator:
    def __init__(self, environment):
```

```

        self.environment = environment

    def clean(self):
        # Clear current position
        self.environment.states[self.environment.agent_location] = "Clean"
        self.environment.performance_score += 1 # Add 1 point for each cleanup
        self.environment.energy_usage += 1 # Clearing movements burn energy

    def move_left(self):
        # The vacuum cleaner moves left to A
        self.environment.agent_location = "A"
        self.environment.energy_usage += 1 # Movements consume energy

    def move_right(self):
        # The vacuum cleaner moves to the right to B
        self.environment.agent_location = "B"
        self.environment.energy_usage += 1 # Movements consume energy

# Defining a Simple Reflective Proxy
class SimpleReflexAgent:
    # Initialisation of objects,
    def __init__(self, environment, actuator):
        self.environment = environment
        self.actuator = actuator

    def act(self):
        # Selecting actions based on surrounding conditions
        if self.environment.is_dirty():
            self.actuator.clean() # Clean up if current position is dirty
            return "Suck"
        elif self.environment.agent_location == "A":
            self.actuator.move_right() # If in A and not dirty, move to B
            return "Right"
        else:
            self.actuator.move_left() # If in B and not dirty, move to A
            return "Left"

# Plotting Functions
def plot_results(ax, steps, scores, energy, title):
    ax.plot(steps, scores, marker="o", label="Performance Score")
    ax.plot(steps, energy, marker="x", label="Energy Usage")
    ax.set_title(title)
    ax.set_xlabel("Time Step")
    ax.set_ylabel("Score / Energy")

```

```

ax.legend()
ax.grid()

# Main function
if __name__ == "__main__":
    # all possible initial configurations
    initial_states = [
        {"A": "Dirty", "B": "Dirty"},
        {"A": "Clean", "B": "Dirty"},
        {"A": "Dirty", "B": "Clean"},
        {"A": "Clean", "B": "Clean"}
    ]

    # all possible vacuum cleaner locations
    initial_locations = ["A", "B"]

    # Plot
    fig, axes = plt.subplots(len(initial_locations), len(initial_states), figsize=(15, 10))
    fig.suptitle("Vacuum World Performance and Energy Usage")

    # Score map generation in the main loop at the same time
    # Enumeration of 8 types (4*2)
    for loc_idx, initial_location in enumerate(initial_locations):
        for state_idx, init_state in enumerate(initial_states):
            print(f"\nInitial State: {init_state}, Initial Location: {initial_location}")

            # Initialize environment and agent location
            environment = VacuumWorld(initial_location=initial_location)
            environment.states = init_state.copy()

            # Initialize actuator and agent
            actuator = Actuator(environment)
            agent = SimpleReflexAgent(environment, actuator)

            # set three lists to store the steps, scores and energy for helping plot diagrams
            # I modeled 4 time steps
            steps, scores, energy = [], [], []
            for step in range(4):

                action = agent.act()

                # Update performance score based on clean squares
                environment.performance_score += sum(1 for state in environment.states.values()
if state == "Clean")

```

```

        steps.append(step + 1)

        scores.append(environment.performance_score)

        energy.append(environment.energy_usage)

    print(f"Step {step + 1}: Action={action},
Location={environment.agent_location}, States={environment.states}, Performance
Score={environment.performance_score}, Energy Usage={environment.energy_usage}")

    # based on the axes[loc_idx, state_idx], steps, scores, energy plot sub-diagrams
    # example: "State: {'A': 'Dirty', 'B': 'Dirty'}, Loc: A"
    plot_results(axes[loc_idx, state_idx], steps, scores, energy, f"State:
{init_state}, Loc: {initial_location}")

# I use tight_layout adjust the layout of sub diagrams, avoiding overlaps.
# [left, bottom, right, top]
plt.tight_layout(rect=[0, 0.03, 1, 0.95])

plt.show()

```

Outcome:

```

Initial State: {'A': 'Dirty', 'B': 'Dirty'}, Initial Location: A
Step 1: Action=Suck, Location=A, States={'A': 'Clean', 'B': 'Dirty'}, Performance Score=2, Energy Usage=1
Step 2: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Dirty'}, Performance Score=3, Energy Usage=2
Step 3: Action=Suck, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=6, Energy Usage=3
Step 4: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=8, Energy Usage=4

Initial State: {'A': 'Clean', 'B': 'Dirty'}, Initial Location: A
Step 1: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Dirty'}, Performance Score=1, Energy Usage=1
Step 2: Action=Suck, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=4, Energy Usage=2
Step 3: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=6, Energy Usage=3
Step 4: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=8, Energy Usage=4

Initial State: {'A': 'Dirty', 'B': 'Clean'}, Initial Location: A
Step 1: Action=Suck, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=3, Energy Usage=1
Step 2: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=5, Energy Usage=2
Step 3: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=7, Energy Usage=3
Step 4: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=9, Energy Usage=4

Initial State: {'A': 'Clean', 'B': 'Clean'}, Initial Location: A
Step 1: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=2, Energy Usage=1
Step 2: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=4, Energy Usage=2
Step 3: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=6, Energy Usage=3
Step 4: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=8, Energy Usage=4

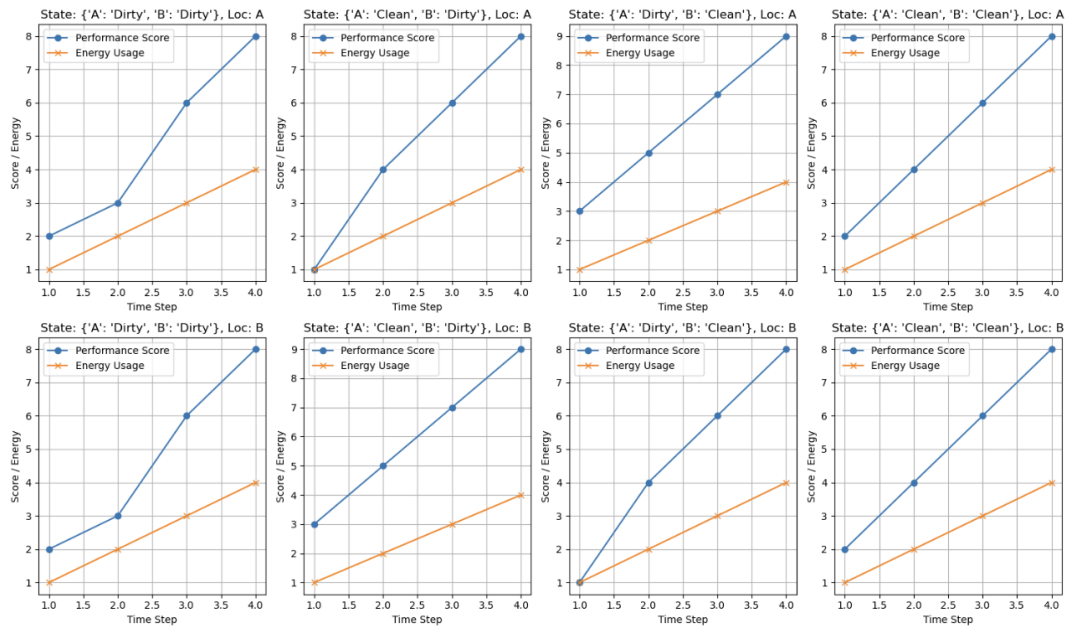
Initial State: {'A': 'Clean', 'B': 'Clean'}, Initial Location: B
Step 1: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=2, Energy Usage=1
Step 2: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=4, Energy Usage=2
Step 3: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=6, Energy Usage=3
Step 4: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=8, Energy Usage=4

Initial State: {'A': 'Clean', 'B': 'Clean'}, Initial Location: B
Step 1: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=2, Energy Usage=1
Step 2: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=4, Energy Usage=2
Step 3: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=6, Energy Usage=3
Step 4: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=8, Energy Usage=4

Initial State: {'A': 'Clean', 'B': 'Clean'}, Initial Location: B
Step 1: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=2, Energy Usage=1
Step 2: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=4, Energy Usage=2
Step 3: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=6, Energy Usage=3
Step 4: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=8, Energy Usage=4

```

Vacuum World Performance and Energy Usage



I choose a case to explain.

Initial State: {'A': 'Dirty', 'B': 'Dirty'}, Initial Location: A
 Step 1: Action=Suck, Location=A, States={'A': 'Clean', 'B': 'Dirty'}, Performance Score=2, Energy Usage=1
 Step 2: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Dirty'}, Performance Score=3, Energy Usage=2
 Step 3: Action=Suck, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=6, Energy Usage=3
 Step 4: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=8, Energy Usage=4

Initial State: {'A': 'Dirty', 'B': 'Dirty'}, Initial Location: A

Description: Both rooms (A and B) start in a dirty state, and the vacuum cleaner initially begins at location A.

Step 1:

- **Action:** The vacuum cleaner performs the Suck action in room A, cleaning the dirty square.
- **State Change:** {'A': 'Clean', 'B': 'Dirty'}.
- **Score:**
 - Cleaning score: **1 point** (room A is cleaned).
 - Clean square score: **1 point** (room A is clean at this time step).
 - Total score for this step: **2 points**.
- **Energy Usage:** 1 (cleaning consumes 1 energy).

Step 2:

- **Action:** The vacuum cleaner moves to room B by performing the Right action.
- **State Change:** {'A': 'Clean', 'B': 'Dirty'} (no change in cleanliness).
- **Score:**
 - Cleaning score: **0 points** (no cleaning performed).
 - Clean square score: **1 point** (room A remains clean).
 - Total score for this step: **1 point**.
- **Energy Usage:** 2 (cumulative, moving consumes 1 energy).

Step 3:

- **Action:** The vacuum cleaner performs the Suck action in room B, cleaning the dirty square.
- **State Change:** {'A': 'Clean', 'B': 'Clean'}.
- **Score:**
 - Cleaning score: **1 point** (room B is cleaned).
 - Clean square score: **2 points** (both A and B are clean).
 - Total score for this step: **3 points**.
- **Energy Usage:** 3 (cumulative, cleaning consumes 1 energy).

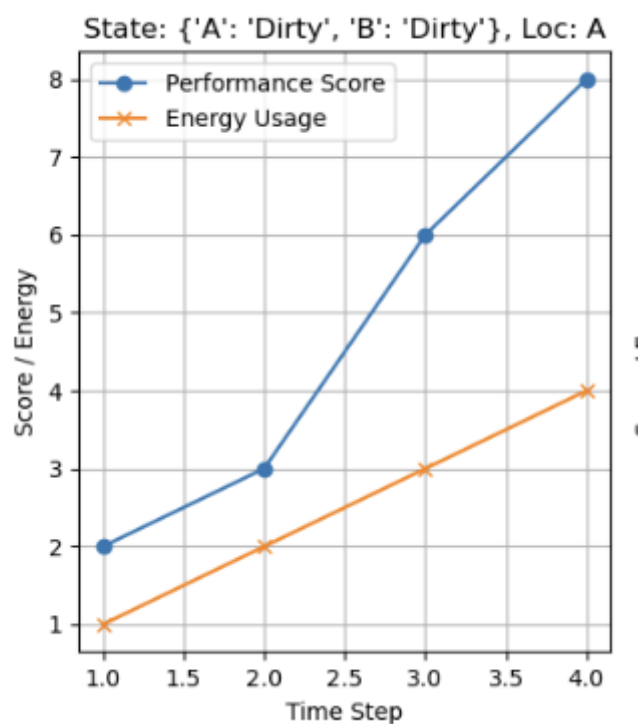
Step 4:

- **Action:** The vacuum cleaner moves back to room A by performing the Left action.
- **State Change:** {'A': 'Clean', 'B': 'Clean'} (no change in cleanliness).
- **Score:**
 - Cleaning score: **0 points** (no cleaning performed).
 - Clean square score: **2 points** (both A and B are clean).
 - Total score for this step: **2 points**.
- **Energy Usage:** 4 (cumulative, moving consumes 1 energy).

Summary:

- **Total Score:** 8 points.
- **Total Energy Usage:** 4.

Based on the case, I draw the diagram. The vacuum cleaner cleans and moves logically, with the score increasing with the number of clean squares in the clean and time step, and the energy increasing by an equal amount with each operation.



Using pygame to simulate conditions

Based on the front code, I want to simulate the different condition directly. So, I searched and found a library called Pygame. I studied and applied it in the following code.

Besides, I set the dirty which will randomly appear in the environment using random library.

```
import pygame
import time
import random

# Initialize pygame
pygame.init()

# Screen dimensions
SCREEN_WIDTH, SCREEN_HEIGHT = 400, 300

# Colors
LIGHT_BLUE = (135, 206, 235)
BLACK = (0, 0, 0)
RED = (255, 0, 0)
LIGHT_RED = (255, 182, 193)
WHITE = (255, 255, 255)

# Initialize screen
screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("Vacuum Cleaner Agent")

# Define rooms
ROOM_A = pygame.Rect(0, 0, SCREEN_WIDTH // 2, SCREEN_HEIGHT)
ROOM_B = pygame.Rect(SCREEN_WIDTH // 2, 0, SCREEN_WIDTH // 2, SCREEN_HEIGHT)

# Draw environment
def draw_environment(environment, agent_location, step, performance_score, energy_usage):
    """Draw the current environment state, agent location, and metrics."""
    screen.fill(LIGHT_BLUE)

    # Draw rooms with colors based on clean/dirty state
    pygame.draw.rect(screen, WHITE if environment.states["A"] == "Clean" else RED, ROOM_A)
    pygame.draw.rect(screen, WHITE if environment.states["B"] == "Clean" else RED, ROOM_B)

    # Draw room borders
    pygame.draw.rect(screen, BLACK, ROOM_A, 2) # Border for Room A
    pygame.draw.rect(screen, BLACK, ROOM_B, 2) # Border for Room B
```



```

# Draw room labels
font = pygame.font.Font(None, 36)
label_a = font.render(f"A: {environment.states['A']}", True, BLACK)
label_b = font.render(f"B: {environment.states['B']}", True, BLACK)
screen.blit(label_a, (ROOM_A.width // 2 - label_a.get_width() // 2, SCREEN_HEIGHT // 2 -
label_a.get_height() // 2))
screen.blit(label_b, (ROOM_B.left + ROOM_B.width // 2 - label_b.get_width() // 2,
SCREEN_HEIGHT // 2 - label_b.get_height() // 2))

# Draw agent
if agent_location == "A":
    agent_pos = (ROOM_A.width // 2, SCREEN_HEIGHT // 2)
else:
    agent_pos = (ROOM_B.left + ROOM_B.width // 2, SCREEN_HEIGHT // 2)
pygame.draw.circle(screen, LIGHT_RED, agent_pos, 20)

# Draw step, performance score, and energy usage
metrics_font = pygame.font.Font(None, 28)
step_text = metrics_font.render(f"Step: {step}", True, BLACK)
performance_text = metrics_font.render(f"Performance: {performance_score}", True, BLACK)
energy_text = metrics_font.render(f"Energy: {energy_usage}", True, BLACK)

screen.blit(step_text, (10, 10))
screen.blit(performance_text, (10, 40))
screen.blit(energy_text, (10, 70))

# Update display
pygame.display.flip()

# Main function to simulate a single scenario
def simulation(steps):
    # Randomly select one of the 8 possible initial configurations
    possible_states = [
        {"A": "Dirty", "B": "Dirty"},
        {"A": "Clean", "B": "Dirty"},
        {"A": "Dirty", "B": "Clean"},
        {"A": "Clean", "B": "Clean"}
    ]
    initial_state = random.choice(possible_states)
    initial_location = random.choice(["A", "B"])

    print(f"Randomly selected initial state: {initial_state}, initial location:
{initial_location}")

```

```

# Initialize environment, actuator, and agent
environment = VacuumWorld(initial_location=initial_location)
environment.states = initial_state
actuator = Actuator(environment)
agent = SimpleReflexAgent(environment, actuator)

# Run simulation for a specified number of steps
for step in range(steps):
    # Handle events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            return

    # Randomly add dirt to either A or B
    if random.random() < 0.3: # 30% chance to add dirt
        dirt_location = random.choice(["A", "B"])
        environment.states[dirt_location] = "Dirty"

    # Agent takes action
    action = agent.act()

    # Update performance score based on clean squares
    environment.performance_score += sum(1 for state in environment.states.values() if
state == "Clean")

    # Draw the environment with metrics
    draw_environment(environment, environment.agent_location, step + 1,
environment.performance_score, environment.energy_usage)

    # Print state info
    print(f"Step {step + 1}: Action={action}, Location={environment.agent_location},
States={environment.states}, Performance Score={environment.performance_score}, Energy
Usage={environment.energy_usage}")

    # Wait for 1 second
    time.sleep(1)

# End simulation
print("Simulation complete.")
time.sleep(2)
pygame.quit()

# Run the simulation

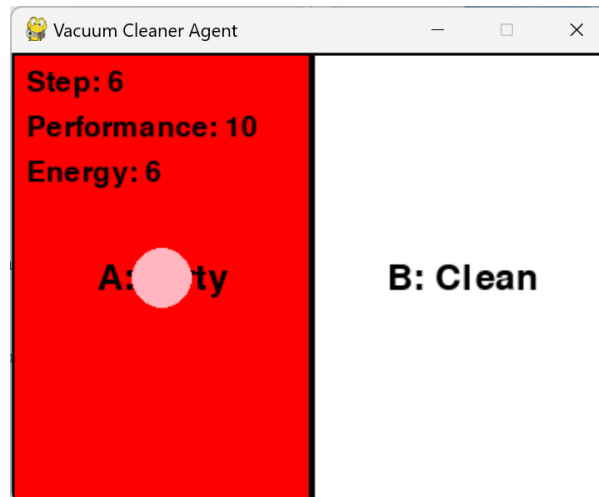
```

```
if __name__ == "__main__":
    user_steps = int(input("Enter the number of steps to simulate: "))
    simulation(user_steps)
```

Run this code, you can choose the number of steps to simulate, and the dirty will randomly appear in the environment using random library. In this case, I choose 20 steps and click 'ENTER' button.

Enter the number of steps to simulate:

You can see the run process step by step.



Simulation process:

```
pygame 2.6.1 (SDL 2.28.4, Python 3.11.5)
Hello from the pygame community. https://www.pygame.org/contribute.html
Enter the number of steps to simulate: 20
Randomly selected initial state: {'A': 'Dirty', 'B': 'Dirty'}, initial location: B
Step 1: Action=Suck, Location=B, States={'A': 'Dirty', 'B': 'Clean'}, Performance Score=2, Energy Usage=1
Step 2: Action=Left, Location=A, States={'A': 'Dirty', 'B': 'Clean'}, Performance Score=3, Energy Usage=2
Step 3: Action=Suck, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=6, Energy Usage=3
Step 4: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Dirty'}, Performance Score=7, Energy Usage=4
Step 5: Action=Suck, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=9, Energy Usage=5
Step 6: Action=Left, Location=A, States={'A': 'Dirty', 'B': 'Clean'}, Performance Score=10, Energy Usage=6
Step 7: Action=Suck, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=13, Energy Usage=7
Step 8: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=15, Energy Usage=8
Step 9: Action=Suck, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=18, Energy Usage=9
Step 10: Action=Suck, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=21, Energy Usage=10
Step 11: Action=Suck, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=24, Energy Usage=11
Step 12: Action=Left, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=26, Energy Usage=12
Step 13: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=28, Energy Usage=13
Step 14: Action=Left, Location=A, States={'A': 'Dirty', 'B': 'Clean'}, Performance Score=29, Energy Usage=14
Step 15: Action=Suck, Location=A, States={'A': 'Clean', 'B': 'Dirty'}, Performance Score=31, Energy Usage=15
Step 16: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Dirty'}, Performance Score=32, Energy Usage=16
Step 17: Action=Suck, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=35, Energy Usage=17
Step 18: Action=Left, Location=A, States={'A': 'Dirty', 'B': 'Clean'}, Performance Score=36, Energy Usage=18
Step 19: Action=Suck, Location=A, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=39, Energy Usage=19
Step 20: Action=Right, Location=B, States={'A': 'Clean', 'B': 'Clean'}, Performance Score=41, Energy Usage=20
Simulation complete.
```